# ZX TURISMO

Explora. Descubre. Reserva.

# MANUAL DE CALIDAD DE SOFTWARE

Documento de Estándares, Procedimientos y Criterios de Calidad

| | |
|---|---|
| **Código del documento:** | ZX-MCS-001 |
| **Versión:** | 1.0.0 |
| **Fecha de elaboración:** | 28/10/2026 |
| **Tipo de proyecto:** | Sitio web turístico (HTML, CSS y JavaScript) |

## EQUIPO DE DESARROLLO

Joel Antonio Guevara
Atilio Efrain Nolasco
Moises Martinez Perez

*Documento técnico interno de referencia de calidad de software*

## TABLA DE CONTENIDO


1. Control del Documento y Versiones — Página 3

2. Política y Objetivos de Calidad — Página 4

3. Estructura Organizacional y Matriz RACI — Página 6

4. Estándares de Codificación — Página 7

5. Estrategia de Pruebas y Control de Errores — Página 11

6. Criterios de Aceptación del Proyecto — Página 14

# 1. CONTROL DEL DOCUMENTO Y VERSIONES

El presente documento constituye el Manual de Calidad de Software del proyecto ZX Turismo. Su propósito es establecer de manera clara y ordenada los procedimientos, estándares técnicos, roles y criterios de evaluación que el equipo de desarrollo aplicará a lo largo del ciclo de vida del proyecto, con el fin de asegurar un producto web funcional, mantenible y orientado a una experiencia de usuario satisfactoria.

El control documental permite rastrear la evolución del manual, identificar responsables de cada cambio y mantener una versión única de referencia para todo el equipo. Cualquier modificación sustancial a este documento debe registrarse en el historial de revisiones antes de considerarse oficial.

## 1.1. Identificación del Documento

| | |
|---|---|
| **Nombre del Proyecto** | ZX Turismo |
| **Código del Documento** | ZX-MCS-001 |
| **Versión Actual** | 1.0.0 |
| **Fecha de Elaboración** | 28/10/2026 |
| **Autores / Integrantes** | Joel Antonio Guevara, Atilio Efrain Nolasco, Moises Martinez Perez |
| **Clasificación** | Documento técnico interno de referencia |

## 1.2. Objetivo del Control Documental

Garantizar que todos los integrantes del equipo trabajen con la misma versión del manual, conozcan los estándares vigentes y puedan consultar de forma rápida las responsabilidades y criterios de aceptación aplicables a cada entrega del proyecto ZX Turismo.

## 1.3. Historial de Revisiones

A continuación se presenta el registro de versiones del documento.

| Versión | Fecha | Descripción del Cambio | Responsable |
|---|---|---|---|
| 0.1.0 | 20/10/2026 | Borrador inicial: estructura general, roles propuestos y primeros estándares de nomenclatura. | Equipo completo |
| 0.2.0 | 24/10/2026 | Incorporación de objetivos medibles, matriz RACI y ejemplos de código orientados a HTML/CSS/JS. | Joel A. Guevara |
| 1.0.0 | 28/10/2026 | Versión completa del manual para uso del equipo. Incluye estrategia de pruebas, casos de prueba y criterios de aceptación. | Equipo completo |

## 2. POLÍTICA Y OBJETIVOS DE CALIDAD

### 2.1. Política de Calidad

El equipo de ZX Turismo asume el compromiso de desarrollar un sitio web turístico que ofrezca a los usuarios una experiencia clara, confiable y agradable. Nos enfocamos en que cada sección del sitio —desde el catálogo de destinos hasta el proceso de reservación de demostración— funcione de manera predecible y sin interrupciones críticas.

Priorizamos la facilidad de navegación y la coherencia visual en todos los dispositivos. El diseño debe adaptarse correctamente a pantallas de escritorio, tabletas y teléfonos móviles, manteniendo legibilidad, contraste adecuado y una estructura de información ordenada. Cualquier elemento interactivo (botones, formularios, filtros y carrito) debe responder de forma inmediata y comunicar claramente su estado al usuario.

Reconocemos la importancia de proteger la información que el usuario introduce en formularios de contacto o reservación. Aunque se trata de un proyecto de demostración, evitamos prácticas inseguras de manipulación de datos y no almacenamos información sensible de manera permanente. El código se mantiene organizado, legible y documentado para facilitar su revisión, mantenimiento y mejora continua.

Finalmente, el equipo se compromete a detectar y corregir defectos de manera sistemática, a realizar pruebas funcionales antes de cada entrega significativa y a considerar la satisfacción del usuario final como el indicador principal de calidad del producto.

### 2.2. Objetivos Medibles de Calidad

Los siguientes objetivos se establecen como metas internas de planificación para orientar el trabajo y evaluar el progreso del proyecto.

| Indicador | Meta Propuesta | Método de Medición | Responsable | Momento de Evaluación |
|---|---|---|---|---|
| Cobertura de pruebas funcionales | ≥ 90% de las funcionalidades principales verificadas | Lista de verificación de casos de prueba ejecutados | Moises M. Perez | Antes de cada entrega formal |
| Errores críticos abiertos | Cero defectos de severidad crítica en la versión de entrega | Revisión del registro de defectos | Equipo completo | Antes de la presentación final |
| Validación de formularios | 100% de los campos obligatorios con validación del lado del cliente | Pruebas manuales de envío con datos inválidos | Atilio E. Nolasco | Al completar cada formulario |
| Funcionamiento del carrito | Cálculo correcto de subtotales y total en el 100% de las pruebas | Casos de prueba de agregar, eliminar y modificar cantidades | Moises M. Perez | Tras implementar la lógica del carrito |

### 2.2. Objetivos Medibles de Calidad (Continuación)

| Indicador | Meta Propuesta | Método de Medición | Responsable | Momento de Evaluación |
|---|---|---|---|---|
| Compatibilidad visual responsiva | Sin roturas de layout en anchos de 320 px, 768 px y 1280 px | Inspección visual en herramientas de desarrollador del navegador | Atilio E. Nolasco | Al finalizar el diseño de cada sección |
| Errores en consola del navegador | Cero errores de JavaScript en el flujo principal de uso | Revisión de la consola (F12) durante las pruebas | Joel A. Guevara | Antes de cada entrega |
| Persistencia del carrito | El carrito mantiene su contenido tras recargar la página (LocalStorage) | Prueba de recarga y verificación de items | Moises M. Perez | Tras implementar LocalStorage |

# 3. ESTRUCTURA ORGANIZACIONAL Y MATRIZ RACI

## 3.1. Roles del Equipo

El proyecto ZX Turismo se desarrolla de manera colaborativa entre tres integrantes. Cada uno asume un rol principal, aunque todos participan en el desarrollo, la revisión de código y las pruebas.

| Integrante | Rol Principal | Áreas de Enfoque |
|---|---|---|
| Joel Antonio Guevara | Coordinador de Proyecto | Planificación, seguimiento, revisión de código, documentación y preparación de la presentación. |
| Atilio Efrain Nolasco | Desarrollador de Interfaz | Diseño visual, HTML semántico, estilos CSS, maquetación responsiva y componentes de interfaz. |
| Moises Martinez Perez | Desarrollador de Funcionalidad y QA | Lógica JavaScript, carrito de compras, validaciones, pruebas funcionales y control de defectos. |

## 3.2. Matriz RACI

La matriz RACI define quién es Responsable de ejecutar cada actividad (R), quién tiene la autoridad final de aprobación (A), quién debe ser Consultado (C) y quién debe ser Informado (I).

**Leyenda:** R = Responsable | A = Aprobador Final | C = Consultado | I = Informado

| Actividad / Entregable | Joel A. Guevara | Atilio E. Nolasco | Moises M. Perez |
|---|---|---|---|
| Recopilación de requisitos | R / A | C | C |
| Planificación del proyecto | R / A | I | I |
| Diseño de la interfaz | C | R / A | - |
| Desarrollo HTML | - | R / A | C |
| Desarrollo CSS | - | R / A | C |
| Programación JavaScript | C | C | R / A |
| Implementación del carrito | - | C | R / A |
| Pruebas de formularios | I | C | R / A |
| Verificación de reservaciones simuladas | C | I | R / A |
| Revisión del código | R / A | R | R |
| Documentación técnica | R / A | C | C |
| Preparación de la presentación | R / A | R | R |
| Validación final del proyecto | A | R | R |

## 4. ESTÁNDARES DE CODIFICACIÓN

Los estándares de este apartado están orientados exclusivamente a las tecnologías utilizadas en ZX Turismo: HTML5, CSS3 y JavaScript (ES6+). El objetivo es mantener un código legible, predecible y fácil de mantener por cualquier integrante del equipo.

### 4.1. Normas de Nomenclatura

**Archivos HTML, CSS y JavaScript:**

- **Archivos HTML:** minúsculas, palabras separadas por guiones. Ejemplo correcto: `destinos.html`, `contacto.html`. Incorrecto: `Destinos.HTML`, `contacto_page.html`.
- **Archivos CSS:** minúsculas con guiones. Ejemplo: `estilos-principales.css`, `componentes.css`.
- **Archivos JavaScript:** minúsculas con guiones o camelCase para módulos. Ejemplo: `carrito.js`, `validaciones-formulario.js`.

**Clases CSS e identificadores HTML:**

- **Clases CSS:** kebab-case (minúsculas con guiones). Ejemplo: `.tarjeta-destino`, `.btn-agregar-carrito`. Evitar: `.TarjetaDestino`, `.btn_agregar`.
- **Identificadores (id):** kebab-case de forma consistente. Ejemplo: `#form-contacto`, `#lista-carrito`.
- **Nombres semánticos:** preferir nombres que describan el propósito (`.seccion-destinos`) sobre nombres genéricos (`.caja1`).

**Variables, funciones y constantes en JavaScript:**

- **Variables y funciones:** camelCase. Ejemplo: `let totalCarrito = 0;` `function calcularSubtotal() {}`.
- **Constantes:** UPPER_SNAKE_CASE. Ejemplo: `const IMPUESTO_IVA = 0.13;` `const MAX_ITEMS_CARRITO = 20;`.
- **Booleanos:** prefijo `is`, `has` o `can`. Ejemplo: `isCarritoVacio`, `hasDescuentoAplicado`.
- **Arrays y colecciones:** plurales. Ejemplo: `const destinos = [];` `const paquetesSeleccionados = [];`.

**Organización de carpetas:**

```
ZX-TURISMO/
├── index.html
├── destinos.html
├── paquetes.html
├── carrito.html
├── contacto.html
├── css/
│   ├── estilos.css
│   └── responsive.css
├── js/
│   ├── main.js
│   ├── carrito.js
│   └── validaciones.js
└── assets/
    ├── img/
    ├── iconos/
    └── fonts/
```

Cada archivo y carpeta tiene un propósito claro: los archivos HTML contienen la estructura de las páginas; la carpeta `css` concentra los estilos; `js` agrupa la lógica de interacción; `assets` almacena recursos estáticos (imágenes e iconos).

### 4.2. Organización del Código

Se aplica el principio de separación de responsabilidades: HTML para la estructura semántica, CSS para la presentación visual y JavaScript para el comportamiento. Se evita el uso de estilos en línea y de scripts embebidos siempre que sea posible.

### 4.3. Ejemplos de Código Conforme a los Estándares

**Ejemplo 1: Tarjeta de destino (HTML)**

```
<!-- Tarjeta de destino turístico -->
<article class="tarjeta-destino" data-id="dest-001">
  <img src="assets/img/destinos/playa-el-tunco.jpg"
       alt="Playa El Tunco, El Salvador"
       class="tarjeta-destino__imagen">
  <div class="tarjeta-destino__contenido">
    <h3 class="tarjeta-destino__titulo">Playa El Tunco</h3>
    <p class="tarjeta-destino__descripcion">
      Destino de surf y atardeceres en la costa salvadoreña.
    </p>
    <p class="tarjeta-destino__precio">Desde $89.00</p>
    <button type="button" class="btn btn-agregar-carrito" data-paquete-id="pkg-001">
      Agregar al carrito
    </button>
  </div>
</article>
```

**Ejemplo 2: Estilos de tarjetas (CSS)**

```
.tarjeta-destino {
  display: flex;
  flex-direction: column;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  background-color: #ffffff;
  transition: transform 0.2s ease;
}

.tarjeta-destino:hover {
  transform: translateY(-4px);
}

.tarjeta-destino__imagen {
  width: 100%;
  height: 180px;
  object-fit: cover;
}

.btn-agregar-carrito {
  background-color: #0A4D68;
  color: #ffffff;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 6px;
  cursor: pointer;
}
```

**Ejemplo 3: Función para calcular subtotal (JavaScript)**

```
/**
 * Calcula el subtotal de un ítem del carrito.
 * @param {number} precioUnitario - Precio del paquete
 * @param {number} cantidad - Unidades seleccionadas
 * @returns {number} Subtotal redondeado a dos decimales
 */
function calcularSubtotal(precioUnitario, cantidad) {
  if (typeof precioUnitario !== "number" || typeof cantidad !== "number") {
    console.error("Parámetros inválidos en calcularSubtotal");
    return 0;
  }
  if (cantidad < 0 || precioUnitario < 0) {
    return 0;
  }
  const subtotal = precioUnitario * cantidad;
  return Math.round(subtotal * 100) / 100;
}
```

**Ejemplo 4: Validación sencilla de formulario (JavaScript)**

```
function validarFormularioContacto(formulario) {
  const nombre = formulario.querySelector("#nombre").value.trim();
  const correo = formulario.querySelector("#correo").value.trim();
  const mensaje = formulario.querySelector("#mensaje").value.trim();
  const errores = [];

  if (nombre.length < 3) {
    errores.push("El nombre debe tener al menos 3 caracteres.");
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(correo)) {
    errores.push("Ingrese un correo electrónico válido.");
  }
  if (mensaje.length < 10) {
    errores.push("El mensaje debe tener al menos 10 caracteres.");
  }

  return {
    esValido: errores.length === 0,
    errores: errores
  };
}
```

### 4.4. Buenas Prácticas de Programación

- ✓ Evitar código duplicado: extraer funciones reutilizables para cálculos, validaciones y manipulación del DOM.
- ✓ Utilizar nombres descriptivos que expresen la intención del código.
- ✓ Separar estructura (HTML), presentación (CSS) y comportamiento (JavaScript).
- ✓ Comentar las secciones complejas o la lógica de negocio no evidente.
- ✓ Validar siempre los datos de entrada del usuario antes de procesarlos.
- ✓ Revisar la consola del navegador para detectar errores de JavaScript antes de cada entrega.
- ✓ Mantener una estructura de carpetas y archivos ordenada y predecible.
- ✓ No almacenar contraseñas ni datos sensibles en LocalStorage. El carrito puede guardar identificadores y cantidades, no información personal.
- ✓ Evitar insertar contenido del usuario directamente con innerHTML sin sanitización. Preferir textContent o createElement cuando sea posible.
- ✓ Usar atributos data-* para asociar información a elementos del DOM de forma semántica.

# 5. ESTRATEGIA DE PRUEBAS Y CONTROL DE ERRORES

## 5.1. Clasificación de Defectos

Los defectos se clasifican según su impacto en la funcionalidad y la experiencia del usuario.

| Severidad | Descripción | Ejemplo en ZX Turismo | Prioridad | Tiempo Objetivo |
|---|---|---|---|---|
| Crítica | Impide el uso principal del sistema o provoca pérdida de datos del carrito. | El carrito calcula un total incorrecto; los botones de reservación no responden. | Inmediata | ≤ 12 horas |
| Alta | Afecta una funcionalidad importante, pero existe una alternativa parcial. | Los filtros de destinos muestran resultados incompletos; el formulario no valida el correo. | Alta | ≤ 24 horas |
| Media | Problema funcional menor o de usabilidad que no bloquea el flujo principal. | Una imagen de destino no carga; el contador del carrito no se actualiza visualmente. | Media | ≤ 48 horas |
| Baja | Defectos cosméticos o de texto que no afectan la funcionalidad. | Texto desalineado; error ortográfico en una etiqueta; espaciado irregular. | Baja | Próxima iteración |

## 5.2. Tipos de Pruebas

**Pruebas de navegación:** Verificar que todos los enlaces del menú y botones conduzcan a las secciones correctas sin errores 404.

**Pruebas del catálogo y filtros:** Confirmar que los destinos se muestran correctamente y los filtros reducen la lista de forma coherente.

**Pruebas del carrito de compras:** Validar agregar, eliminar, modificar cantidades y calcular subtotales y totales.

**Pruebas de persistencia (LocalStorage):** Comprobar que el carrito conserva su contenido tras recargar la página.

**Pruebas de formularios y reservaciones:** Asegurar validaciones del lado del cliente y flujo de reservación simulada sin errores.

**Pruebas de compatibilidad y responsividad:** Verificar adaptación de layout a distintos anchos de pantalla (320 px, 768 px, 1280 px).

**Pruebas de consola y accesibilidad básica:** Detectar errores de JavaScript en consola y verificar atributos alt en imágenes.

### 5.3. Casos de Prueba

Registro de casos de prueba representativos para la verificación funcional del sistema.

| ID | Funcionalidad / Pasos | Resultado Esperado | Criterio de Aprobación | Estado |
|---|---|---|---|---|
| CP-01 | Navegación menú: 1. Abrir index.html 2. Clic en cada enlace | Cada sección carga correctamente | Sin enlaces rotos | Pendiente |
| CP-02 | Visualización catálogo: 1. Ir a destinos 2. Verificar tarjetas | Se muestran al menos 4 destinos completos | Contenido completo | Pendiente |
| CP-03 | Filtro por región: 1. Seleccionar filtro Costa 2. Observar | Solo destinos de la costa aparecen | Filtro correcto | Pendiente |
| CP-04 | Agregar al carrito: 1. Clic en Agregar 2. Abrir carrito | El paquete aparece con cantidad 1 | Ítem añadido | Pendiente |
| CP-05 | Eliminar del carrito: 1. Tener items 2. Clic en eliminar | El item desaparece y el total se recalcula | Eliminación correcta | Pendiente |
| CP-06 | Cálculo de total: 1. Agregar 2 paquetes ($50 y $80) | Total = $130.00 | Cálculo exacto | Pendiente |
| CP-07 | Persistencia: 1. Agregar items 2. Recargar página | Los mismos ítems permanecen | Datos persistentes | Pendiente |
| CP-08 | Validación formulario: 1. Enviar vacío / datos inválidos | Se muestran mensajes de error apropiados | Validación activa | Pendiente |
| CP-09 | Reservación: 1. Datos válidos 2. Confirmar | Mensaje de confirmación visible | Flujo completo | Pendiente |
| CP-10 | Vista móvil: 1. DevTools 320px 2. Navegar | Sin scroll horizontal ni elementos cortados | Layout adaptado | Pendiente |
| CP-11 | Consola: 1. Abrir DevTools 2. Recorrer flujo | Ningún error de JavaScript en rojo | Consola limpia | Pendiente |
| CP-12 | Atributos alt: 1. Inspeccionar imágenes de destinos | Imágenes relevantes con alt descriptivo | Accesibilidad básica | Pendiente |

### 5.4. Ciclo de Vida de los Errores

Todo defecto detectado debe seguir el siguiente flujo de estados. Si una corrección no resuelve el problema, el defecto regresa al estado "En corrección" o "En análisis" según corresponda.

**Detectado → Registrado → En análisis → En corrección → Pendiente de verificación → Cerrado**

- **Detectado:** Cualquier integrante identifica un comportamiento anómalo durante el desarrollo o pruebas.
- **Registrado:** Se documenta el defecto con descripción, pasos para reproducirlo, severidad y capturas si es necesario.
- **En análisis:** El responsable de la funcionalidad afectada evalúa la causa y estima el esfuerzo de solución.
- **En corrección:** Se implementa la solución correspondiente en el código fuente.
- **Pendiente de verificación:** Otro integrante prueba que el defecto ya no se reproduce.
- **Cerrado:** Se confirma la solución y se actualiza el registro. Si el defecto reaparece, se reabre.

# 6. CRITERIOS DE ACEPTACIÓN DEL PROYECTO

Un módulo o la versión completa de ZX Turismo se considerará aceptable para entrega cuando cumpla los criterios obligatorios listados a continuación. Las excepciones o defectos menores pendientes deben registrarse de forma explícita y no deben incluir severidad crítica o alta.

## 6.1. Lista de Verificación

**Estructura y organización:**

- ✓ La estructura de carpetas sigue la convención definida (HTML en raíz o páginas, css/, js/, assets/).
- ✓ Los nombres de archivos y clases cumplen las normas de nomenclatura del presente manual.
- ✓ No existen estilos en línea ni scripts embebidos innecesarios en el código de producción.

**Funcionalidad principal:**

- ✓ Todos los enlaces del menú de navegación funcionan y dirigen a la sección correcta.
- ✓ Las tarjetas de destinos y paquetes se visualizan con imagen, título, descripción breve y precio.
- ✓ Los filtros de búsqueda o categorización reducen correctamente el listado de resultados.
- ✓ Es posible agregar paquetes al carrito, modificar cantidades y eliminar ítems.
- ✓ El subtotal y el total del carrito se calculan de forma correcta en todas las pruebas realizadas.
- ✓ El contenido del carrito persiste tras recargar la página (LocalStorage) cuando esta funcionalidad está implementada.
- ✓ Los formularios validan campos obligatorios y formatos (correo, longitud mínima) antes de permitir el envío.
- ✓ El flujo de reservación de demostración muestra una confirmación clara al usuario.

**Calidad técnica y presentación:**

- ✓ El diseño se adapta sin roturas graves a anchos de 320 px, 768 px y 1280 px.
- ✓ No existen errores críticos de JavaScript en la consola durante el flujo principal de uso.
- ✓ Las imágenes relevantes incluyen atributo alt descriptivo.
- ✓ El código ha sido revisado por al menos un integrante distinto al autor de cada módulo principal.
- ✓ La documentación técnica se encuentra actualizada.
- ✓ No quedan defectos de severidad crítica o alta sin resolver en la versión de entrega.

### 6.2. Registro de Excepciones

Si al momento de la entrega existen defectos de severidad media o baja que el equipo decide no corregir de inmediato, estos deben listarse en un apartado de observaciones con: identificador del defecto, descripción breve, severidad y justificación. Ninguna excepción de severidad crítica o alta será aceptable para considerar el proyecto aprobado según este manual.

### 6.3. Conformidad del Equipo

Este Manual de Calidad de Software para el proyecto ZX Turismo ha sido elaborado de manera colaborativa por los integrantes del equipo con el propósito de guiar el desarrollo, las pruebas y la evaluación de la calidad del sitio web. La versión del proyecto que se presente formalmente deberá contar con el acuerdo de los tres integrantes respecto al cumplimiento de los criterios obligatorios.

| Integrante | Fecha | Estado |
|---|---|---|
| Joel Antonio Guevara | 28/10/2026 | Conforme |
| Atilio Efrain Nolasco | 28/10/2026 | Conforme |
| Moises Martinez Perez | 28/10/2026 | Conforme |

**Fin del Manual de Calidad de Software ZX-MCS-001**